

# Statistical Programming Languages

## Exercises: Day 2

### Reading and Writing Data

#### Exercise 1 (A simple data frame):

Create a `data.frame` with the data in the following table and solve the problems a)–b)

| Country    | Index | Unemployment |
|------------|-------|--------------|
| Belgium    | 2.8   | 9.4          |
| Denmark    | 1.2   | 10.4         |
| France     | 2.1   | 10.8         |
| GB         | 1.6   | 10.5         |
| Ireland    | 1.5   | 18.4         |
| Italy      | 4.6   | 11.1         |
| Luxembourg | 3.6   | 2.6          |
| Holland    | 2.1   | 8.8          |
| Portugal   | 6.5   | 5.0          |
| Spain      | 4.6   | 21.5         |
| USA        | 3.0   | 6.7          |
| Japan      | 1.3   | 2.5          |
| Germany    | 4.2   | 5.6          |

- Compute the minimum and maximum of each column, print the values and their corresponding country.
- Compute the range of each column, given by  $\text{range}(\mathbf{x}) := \max(\mathbf{x}) - \min(\mathbf{x})$ . (Hint: `?range`)

#### Exercise 2 (Manipulate data):

Load the R dataset `mtcars` and figure out the variables using `help()`. Afterwards solve the following tasks:

- Create a data set `mtcars_mod`, which corresponds to the data set `mtcars` sorted by columns `mpg` and `cyl` in descending order.
- Remove the column `carb` from `mtcars_mod`.
- Switch the columns `mpg` and `hp` in `mtcars_mod`.
- Extract only the cars of brand Mercedes by finding the indices of row names containing the string "Merc". (Hint: `?grep`)

**Exercise 3 (Read and write data):**

Download the file `dax_prices.csv` from the HU Box folder and save it to your current working directory.

- Read the file `dax_prices.csv` into a data frame `dax.prices`.
- Inspect the first rows of `dax.prices`. Are the prices (contained in variable `DAX`) correctly interpreted as numeric?
- Rename the column `DAX` to `DAX.Prices`.
- Write `dax.prices` to a file named `dax_prices.txt` in your working directory, with “;” as separator and “.” as decimal point.
- Read `dax_prices.txt` into a data frame `dax.prices.txt`. Make sure the prices are correctly interpreted as numeric.

**String Handling****Exercise 4 (Regular expressions I):**

Create a regular expression that matches all the items in the first column (positive examples) but none of those in the second (negative examples).

a)

| Positive | Negative |
|----------|----------|
| pit      | pt       |
| spot     | Pot      |
| spate    | peat     |
| slap two | part     |
| respice  |          |

```
a <- c("pit", "spot", "spate", "slap two", "respice", "pt", "Pot", "peat",
      "part")
```

b)

| Positive   | Negative   |
|------------|------------|
| rap them   | aleht      |
| tapeth     | happy them |
| apth       | tarpth     |
| wrap/try   | Apt        |
| sap tray   | peth       |
| 87ap9th    | tarreth    |
| apothecary | ddapdg     |
|            | apples     |
|            | shape the  |

```
b <- c("rap them", "tapeth", "apth", "wrap/try", "sap tray", "87ap9th",
      "apothecary", "aleht", "happy them", "tarpth", "Apt", "peth", "tarreth",
      "ddapdg", "apples", "shape the")
```

c)

| Positive   | Negative |
|------------|----------|
| affgfkking | fgok     |
| rafgkahe   | a fgk    |
| bafghk     | affgm    |
| baffgkit   | afffhk   |
| baffg kit  | afg.K    |
|            | aff gm   |
|            | afffhgk  |

```
c <- c("affgfkking", "rafgkahe", "bafghk", "baffgkit", "baffg kit", "fgok",
"a fgk", "affgm", "afffhk", "afg.K", "aff gm", "afffhgk")
```

d)

| Positive                                                                                                                                                                                                        | Negative                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| assumes word senses. Within<br>does the clustering. In the<br>but when? It was hard to tell<br>he arrive." After she had<br>mess! He did not let it<br>it wasn't hers!' She replied<br>always thought so.) Then | in the U.S.A., people often<br>John?"', he often thought, but<br>weighed 17.5 grams<br>well ... they'd better not<br>A.I. has long been a very<br>like that", he thought<br>but W. G. Grace never had much |

```
d <- c("assumes word senses. Within", "does the clustering. In the",
"but when? It was hard to tell", 'he arrive." After she had',
"mess! He did not let it", "it wasn't hers!' She replied",
"always thought so.) Then", "in the U.S.A., people often",
' John?', he often thought, but', "weighed 17.5 grams",
"well ... they'd better not", 'A.I. has long been a very',
'like that", he thought', "but W. G. Grace never had much")
```

Source of Exercise 1: <https://regex.sketchengine.co.uk/>

### Exercise 5 (Regular expressions II):

- Create a regular expression that matches the integers from 0 to 255.
- Create a regular expression that matches a decimal number.

### Exercise 6 (Regular expressions III):

Create two regular expressions that find all date and time formats, respectively, from the following list of date and time formats.

```

05/29/2015
Friday, 29 May 2015
Friday, 29 May 2015 05:50
Friday, 29 May 2015 05:50 AM
Friday, 29 May 2015 5:50
Friday, 29 May 2015 5:50 AM
Friday, 29 May 2015 05:50:06
05/29/2015 05:50
05/29/2015 05:50 AM
05/29/2015 5:50
05/29/2015 5:50 AM
05/29/2015 05:50:06
2015-05-16T05:50:06.7199222-04:00
Fri, 16 May 2015 05:50:06 GMT
2015-05-16T05:50:06
05:50
05:50 AM
5:50
5:50 AM
05:50:06

dt <- c("05/29/2015", "Friday, 29 May 2015", "Friday, 29 May 2015 05:50",
        "Friday, 29 May 2015 05:50 AM", "Friday, 29 May 2015 5:50",
        "Friday, 29 May 2015 5:50 AM", "Friday, 29 May 2015 05:50:06",
        "05/29/2015 05:50", "05/29/2015 05:50 AM", "05/29/2015 5:50",
        "05/29/2015 5:50 AM", "05/29/2015 05:50:06",
        "2015-05-16T05:50:06.7199222-04:00", "Fri, 16 May 2015 05:50:06 GMT",
        "2015-05-16T05:50:06", "05:50", "05:50 AM", "5:50", "5:50 AM",
        "05:50:06")

```

### Exercise 7 (Regular expressions IV):

Search in the internet for regular expressions to extract an HTML tag. Do they differ from the one used in the slides? If yes, what is the difference?